

Full Stack Development with MERN – Project Documentation

1. Introduction

Project Title

Implement Client Script and UI Policy (Incident)

Team Members

- Vanalakshmi J
- Devatharshini R
- Lavanya S

2. Project Overview

Purpose

The **Implement Client Script and UI Policy (Incident)** project is designed to demonstrate how client-side scripts and UI policies can be used to control and improve an Incident Management interface.

The project focuses on making the incident form more interactive and user-friendly. Client Scripts are used to perform actions based on user input, while UI Policies are used to dynamically control the visibility, mandatory status, and read-only state of form fields.

Features

The main features of the project are:

- Incident creation and management
- Incident form with different fields
- Client-side form validation

- Dynamic field visibility
- Making fields mandatory based on conditions
- Making fields read-only when required
- Automatic field value changes
- Incident priority handling
- Incident state management
- User-friendly interface
- Backend API for incident data
- MongoDB database for storing incidents

3. Architecture

The project follows the **MERN Stack architecture**, consisting of MongoDB, Express.js, React.js, and Node.js.

Frontend – React.js

React.js is used to create the Incident Management user interface.

The frontend contains:

- Incident form
- Incident list
- Incident details
- Priority and state fields
- Dynamic form controls
- Validation messages

Client-side logic is implemented using JavaScript/React event handlers to respond to user actions.

Backend – Node.js and Express.js

Node.js and Express.js are used to develop the backend server.

The backend is responsible for:

- Creating incidents
- Retrieving incidents
- Updating incidents
- Deleting incidents
- Validating incoming data
- Communicating with MongoDB

Database – MongoDB

MongoDB is used to store incident information.

A typical Incident document may contain:

Incident

```
|— incidentNumber
|— shortDescription
|— description
|— caller
|— category
|— priority
|— state
|— assignmentGroup
|— createdAt
```

4. Setup Instructions

Prerequisites

The following software is required:

- Node.js
- npm
- MongoDB
- Visual Studio Code
- Web Browser
- Postman (optional)

Installation

Step 1: Install Node.js and MongoDB.

Step 2: Open the project folder in Visual Studio Code.

Step 3: Install frontend dependencies:

```
cd client  
npm install
```

Step 4: Install backend dependencies:

```
cd server  
npm install
```

Step 5: Configure the MongoDB connection in the `.env` file.

Example:

```
PORT=5000  
MONGO_URI=your_mongodb_connection_string
```

Step 6: Start the MongoDB service.

Step 7: Start the frontend and backend servers.

5. Folder Structure

Client

```
client/  
├── public/  
├── src/  
│   ├── components/  
│   │   ├── IncidentForm.js  
│   │   ├── IncidentList.js  
│   │   └── IncidentDetails.js
```

```
|
|
| └─ pages/
|   └─ IncidentPage.js
|
| └─ services/
|   └─ incidentService.js
|
| └─ App.js
|   └─ index.js
|
└─ package.json
```

Server

```
server/
|
| └─ models/
|   └─ Incident.js
|
| └─ routes/
|   └─ incidentRoutes.js
|
| └─ controllers/
|   └─ incidentController.js
|
| └─ config/
|   └─ db.js
|
| └─ server.js
| └─ .env
└─ package.json
```

6. Running the Application

Frontend

```
cd client
npm start
```

The frontend will run on:

<http://localhost:3000>

Backend

Open another terminal:

```
cd server
npm start
```

The backend will run on:

<http://localhost:5000>

7. API Documentation

The backend provides REST APIs for incident management.

Method	Endpoint	Description
GET	/api/incidents	Get all incidents
GET	/api/incidents/:id	Get a specific incident
POST	/api/incidents	Create a new incident
PUT	/api/incidents/:id	Update an incident
DELETE	/api/incidents/:id	Delete an incident

Example Request

POST /api/incidents

Example Response

```
{
  "success": true,
  "message": "Incident created successfully",
  "incident": {
    "incidentNumber": "INC001",
    "shortDescription": "System not working",
    "priority": "High",
    "state": "New"
  }
}
```

8. Authentication

Authentication can be implemented to ensure that only authorized users can access and manage incident records.

The system can use **JWT (JSON WebToken)** for authentication.

The authentication process is:

1. User enters login credentials.
2. Frontend sends credentials to the backend.
3. Backend verifies the credentials.
4. JWT token is generated.
5. The token is used to access protected incident APIs.
6. Unauthorized users are prevented from accessing restricted operations.

9. User Interface

The application provides an interactive Incident Management interface.

Incident Form

The incident form contains fields such as:

- Incident Number
- Short Description
- Description
- Caller
- Category
- Priority
- State
- Assignment Group

Client Script

Client-side scripts perform actions based on user input.

For example:

- Validate the incident description.
- Automatically change a field value.
- Display an alert or validation message.
- Update fields based on user selection.

UI Policy

UI Policies dynamically control the form fields.

For example:

- Make **Description** mandatory for certain incidents.
- Make fields read-only when the incident is closed.
- Show or hide fields based on selected values.
- Change field behavior according to incident state or priority.

10. Testing

Testing is performed to verify that the Incident Management application works correctly.

Testing Types

- **Functional Testing** – Tests incident creation, updating, and deletion.
- **UI Testing** – Checks the user interface and form behavior.
- **Client Script Testing** – Verifies that scripts respond correctly to user actions.
- **UI Policy Testing** – Checks field visibility, mandatory status, and read-only behavior.
- **API Testing** – Tests backend REST APIs.
- **Database Testing** – Verifies that incident data is correctly stored in MongoDB.

Testing Tools

- Browser Developer Tools
- Postman
- MongoDB Compass
- Visual Studio Code

11. Screenshots or Demo

The following screenshots can be included:

1. Login Page
2. Incident Management Dashboard
3. Create Incident Form
4. Incident List
5. Client Script Working
6. UI Policy Working
7. Incident Priority Selection
8. Incident State Change

- 9. Updated Incident Details
- 10. MongoDB Incident Collection

Demo Link: [Add your project/demo link here]

12. Known Issues

Some possible limitations of the current project are:

- Advanced role-based permissions may not be implemented.
- Some validation rules may need additional improvements.
- Large numbers of incidents may require pagination.
- Real-time incident notifications are not included.
- Advanced reporting features may be added in future versions.

13. Future Enhancements

The project can be enhanced with the following features:

- Role-based access control
- Real-time incident notifications
- Email notifications
- Incident assignment to support groups
- Incident history tracking
- Advanced search and filtering
- Incident analytics dashboard
- Priority-based reporting
- SLA tracking
- Automatic incident escalation
- Export incident reports as PDF/Excel
- Mobile-friendly interface
- AI-based incident classification

Conclusion

The **Implement Client Script and UI Policy (Incident)** project demonstrates how dynamic form behavior can improve an Incident Management system. Client Scripts help perform actions and validations based on user interactions, while UI Policies dynamically control the behavior of form fields. By combining these features with the MERN stack, the project provides an interactive and efficient platform for managing incidents.